

1. Reading the data

```
def getTextLinesFromFile(fileName):
    ret = []
    with open(fileName, "r") as file:
        lines = file.readlines()
        for line in lines:
            ret.append(line.strip())
    return ret
```

```
lines = getTextLinesFromFile("dataset.txt")
print(len(lines))
```

```
4230
```

2. Substitute course codes.

```
substitutions = dict()
substitutes = getTextLinesFromFile("substitutions.txt")
for line in substitutes:
    oldCC, newCC = line.split("\t")
    substitutions[oldCC.upper()] = newCC
```

```
def getSubstituteCode(oldCC):
    return substitutions[oldCC.upper()]
```

```
-----
FileNotFoundError                                Traceback (most recent call last)
/tmp/ipython-input-1061231950.py in <cell line: 0>()
      1 substitutions = dict()
----> 2 substitutes = getTextLinesFromFile("substitutions.txt")
      3 for line in substitutes:
      4     oldCC, newCC = line.split("\t")
      5     substitutions[oldCC.upper()] = newCC

/tmp/ipython-input-2617913689.py in getTextLinesFromFile(fileName)
      1 def getTextLinesFromFile(fileName):
      2     ret = []
----> 3     with open(fileName, "r") as file:
      4         lines = file.readlines()
      5         for line in lines:

FileNotFoundError: [Errno 2] No such file or directory: 'substitutions.txt'
```

```
for i in range(1, len(lines)):
    print(lines[i].split("\t")[2])
    oldCC = lines[i].split("\t")[2]
    newCC = getSubstituteCode(oldCC)
    lines[i] = lines[i].replace(oldCC, newCC)
    print(lines[i].split("\t")[2])
    #print(getSubstituteCode(lines[i].split("\t")[0]))
```

20191CCE0081

```

-----
NameError                                Traceback (most recent call last)
/tmp/ipython-input-399875488.py in <cell line: 0>()
      2 print(lines[i].split("\t")[2])
      3 oldCC = lines[i].split("\t")[2]
----> 4 newCC = getSubstituteCode(oldCC)
      5 lines[i] = lines[i].replace(oldCC, newCC)
      6 print(lines[i].split("\t")[2])

```

NameError: name 'getSubstituteCode' is not defined

Double-click (or enter) to edit

```

MOOC_COURSES = {
    "BBB2016",
    "CSE3111",
    "CSE3202",
    "MGT3001",
    "ENG3004",
    "MEC3301",
    "CHE1001",
    "CHE1018",
    "CHE1020"
}

```

```

# Count number of students per course
course_counts = {}

for i in range(1, len(lines)):
    entries = lines[i].split("\t")
    courseCode = entries[2]

    if courseCode in course_counts:
        course_counts[courseCode] += 1
    else:
        course_counts[courseCode] = 1

```

```

filtered_lines = [lines[0]] # keep header

for i in range(1, len(lines)):
    entries = lines[i].split("\t")
    courseCode = entries[2]

    # Skip / Remove if MOOC
    if courseCode in MOOC_COURSES:
        continue

    # Skip / Remove if fewer than 8 students
    if course_counts.get(courseCode, 0) < 21:
        continue

```

```

        filtered_lines.append(lines[i])

# Replace the original lines with the filtered version
lines = filtered_lines

```

3. Create a list of course codes registered for each roll number. Use a dictionary for representing the roll numbers. The key is the roll number, the value is the list of course codes associated with the roll number.

```

registrations = dict()
for i in range(1, len(lines)):
    entries = lines[i].split("\t")
    rollNo = entries[0]
    courseCode = entries[2]
    if rollNo in registrations:
        registrations[rollNo].append(courseCode)
    else:
        registrations[rollNo] = [courseCode]
print(len(registrations))
print(registrations)

```

```

1228
{'20191CCE0081': ['CSE2013', 'CSE2067', 'CSE3150'], '20191ISE0109': ['CSE2013', 'CSE3078'], '20191IST001': ['CSE2013', 'CSE3078', 'CSE3150']}

```

```

for rollNo in registrations:
    print(rollNo, registrations[rollNo])

```

[Show hidden output](#)

```

from collections import Counter

def getDataFromRegistrations(registrations):
    ret = []
    for rollNo in registrations:
        ret.append(registrations[rollNo])
    return ret

def count_distinct_lists(list_of_lists):
    """Count occurrences of distinct lists, treating unsorted lists as equal"""
    # Convert each list to a sorted tuple for comparison
    sorted_tuples = [tuple(sorted(lst)) for lst in list_of_lists]
    return Counter(sorted_tuples)

# Example usage
data = getDataFromRegistrations(registrations)

result = count_distinct_lists(data)
print(result)

Counter({('CSE1006',): 52, ('MAT1002',): 32, ('CSE3151',): 30, ('CSE1001',): 27, ('CSE2001',): 26, ('E001',): 25})

```

```

print(len(result))

```

640

```

users = dict()
for i in range(1, len(lines)):
    entries = lines[i].split("\t")
    rollNo = entries[0]
    courseCode = entries[2]
    if courseCode in users:
        users[courseCode].add(rollNo)
    else:
        users[courseCode] = {rollNo}
print(len(users))
print(users)

```

49

```
{'CSE2013': {'20211CSD0022', '20211CSD0138', '20211CSE0458', '20211CSE0741', '20191ISE0109', '20211CSE0
```

```

for user in users:

    print(user, len(users[user]))

```

[Show hidden output](#)

Start coding or [generate](#) with AI.

```

def min_groups(users):
    """
    users: dict(name -> set_of_items)
    returns: list of groups, each group is a list of user names
    """

    names = list(users.keys())

    # Build conflict graph
    conflict = {name: set() for name in names}

    for i in range(len(names)):
        for j in range(i + 1, len(names)):
            u, v = names[i], names[j]
            # If share common items, they conflict
            if set(users[u]) & set(users[v]):
                conflict[u].add(v)
                conflict[v].add(u)

    # Greedy coloring
    color = {} # user -> group number
    for u in sorted(names, key=lambda x: len(conflict[x]), reverse=True):
        assigned_colors = {color[v] for v in conflict[u] if v in color}

        # choose smallest available color
        c = 0
        while c in assigned_colors:
            c += 1

```

```

        color[u] = c

    # Convert colors to group list
    groups = {}
    for user, c in color.items():
        groups.setdefault(c, []).append(user)

    return list(groups.values())

```

```

groups = min_groups(users)
print(groups)
print(len(groups))

```

```

[['ECE2007', 'CSE3078', 'CSE2010', 'CSE3150'], ['MAT1002', 'CSE3082', 'CSE2015', 'PPS1012'], ['CSE2014',
15

```

```

days=["Monday","Tuesday","Wednesday","Thursday","Friday"]
time_slots=[
    "9:00am-11:00am",
    "11:00am-1:00pm",
    #"1:00pm-2:00pm(Lunch Break)",
    "2:00pm-4:00pm"
]
def getslot(i):
    day=days[i//len(days)]
    time=time_slots[i%len(time_slots)]
    return day+" "+time+"\t"

```

```

for i in range(len(groups)):
    print(getslot(i),":",groups[i])

```

```

Monday 9:00am-11:00am : ['ECE2007', 'CSE3078', 'CSE2010', 'CSE3150']
Monday 11:00am-1:00pm : ['MAT1002', 'CSE3082', 'CSE2015', 'PPS1012']
Monday 2:00pm-4:00pm : ['CSE2014', 'MAT2003', 'CIV1008', 'DES1146']
Monday 9:00am-11:00am : ['CSE2007', 'MEC1006']
Monday 11:00am-1:00pm : ['CSE2009', 'MAT1003', 'PPS4006']
Tuesday 2:00pm-4:00pm : ['CSE2001', 'CSE3151', 'CSE1006', 'MGT2021']
Tuesday 9:00am-11:00am : ['MGT2020', 'MAT1001', 'EEE1007']
Tuesday 11:00am-1:00pm : ['CSE2018', 'PPS4002', 'ECE1001', 'ECE2010']
Tuesday 2:00pm-4:00pm : ['CSE2074', 'PPS1001', 'CSE2013']
Tuesday 9:00am-11:00am : ['CSE1005', 'CSE2067', 'CSE3075', 'LAW1007', 'CSE3348']
Wednesday 11:00am-1:00pm : ['CSE2011', 'ENG2001', 'PPS4005']
Wednesday 2:00pm-4:00pm : ['CSE2021', 'CSE2066', 'CSE1004']
Wednesday 9:00am-11:00am : ['MAT2004', 'ECE2011', 'PHY1002']
Wednesday 11:00am-1:00pm : ['CSE1001', 'CSE3001']
Wednesday 2:00pm-4:00pm : ['PPS1011', 'CSE2027']

```

Start coding or [generate](#) with AI.

